Tutorial/Laboratory 06

Form Processing with HTML5, Bootstrap and PHP

1. LEARNING OUTCOMES

Upon completion of these laboratory exercises, you should be able to:

- Create responsive, user-friendly forms for collecting input.
- Implement a comprehensive strategy for validating & sanitizing form input utilizing a combination of client-side and server-side techniques.

2. SOFTWARE

2.1 Required:

- Visual Studio Code (https://code.visualstudio.com/)
- PHP 8.2 (or later) (https://www.php.net/downloads)

2.2 Recommended:

- PHP Intelephense extension for VS Code (https://marketplace.visualstudio.com/items?itemName=bmewburn.vscode-intelephense-client)

3. EXERCISE 1: ADD A MEMBER REGISTRATION FORM

3.1 In this exercise, we'll add a form that will allow users to register as World of Pets members.

3.2 Make a copy of your latest World of Pets project from the previous Lab and add a new PHP page called `register.php`. You'll also want to add a menu item to your `<nav>` element that links to `register.php`. If you wish, you can use icons for your Register (and Login, which we'll add in a future exercise), such as the Google Material Design icons shown in the screenshots in this document.

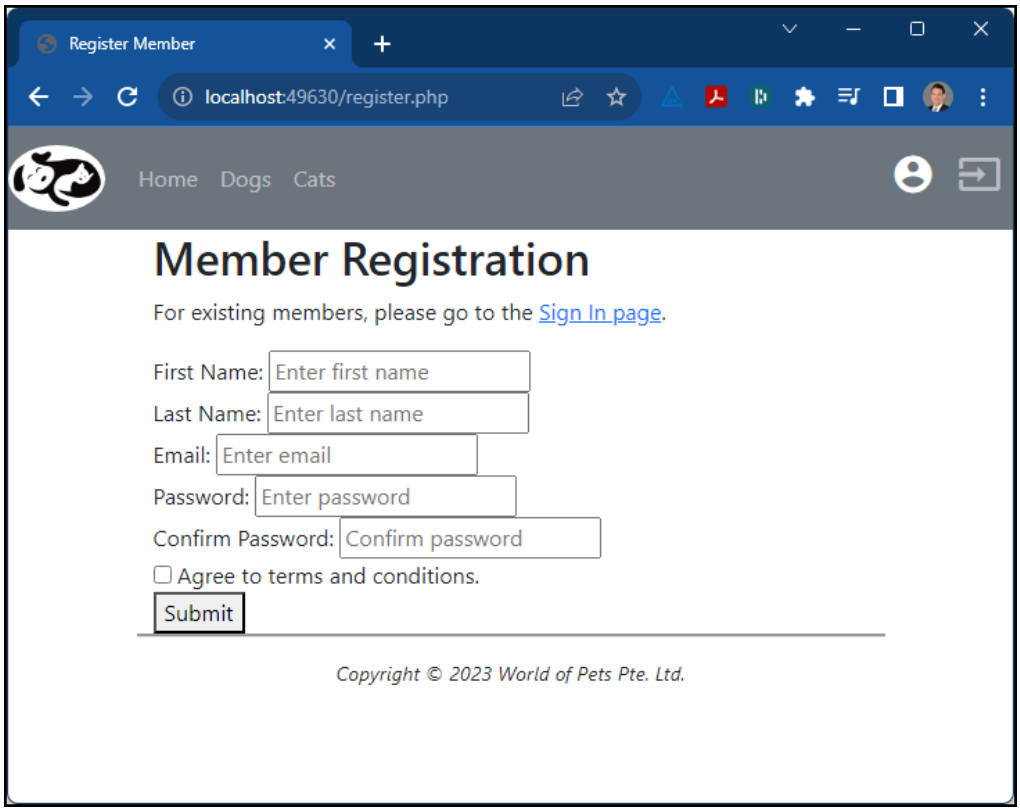
3.3 Edit `register.php` to add the `<head>` element (you can copy from `index.php` or use a PHP include), then add the following `<body>` element:

```
<body>
    <?php
    include "inc/nav.inc.php";
    ?>

    <main class="container">
        <h1>Member Registration</h1>
        <p>
            For existing members, please go to the
            <a href="#">Sign In page</a>.
        </p>
        <form action="process_register.php" method="post">
            <label for="fname">First Name:</label>
            <input type="text" id="fname" name="fname"
                   placeholder="Enter first name">
            <br>
            <label for="lname">Last Name:</label>
            <input type="text" id="lname" name="lname"
                   placeholder="Enter last name">
            <br>
            <label for="email">Email:</label>
            <input type="email" id="email" name="email"
                   placeholder="Enter email">
            <br>
            <label for="pwd">Password:</label>
            <input type="password" id="pwd" name="pwd"
                   placeholder="Enter password">
            <br>
            <label for="pwd_confirm">Confirm Password:</label>
            <input type="password" id="pwd_confirm" name="pwd_confirm"
                   placeholder="Confirm password">
            <br>
            <label>
                <input type="checkbox" name="agree">
                Agree to terms and conditions.
            </label>
            <br>
            <button type="submit">Submit</button>
        </form>
    </main>

    <?php
    include "inc/footer.inc.php";
    ?>
</body>
```

3.4 Follow the procedure from the previous lab exercise to set up the `launch.json` file and run your PHP application. Navigate to the Member Registration page and you should see a basic HTML5 form without any bells or whistles:



This form is semantically correct and accessible, but it is not responsive nor very presentable, so lets' fix that using our old friend, Bootstrap.

3.5 Following are the general steps to "Bootstrapify" our form in order to make it responsive and more attractive (the code is given below):

a. Remove the `<br>` tags and wrap all input groups with elements (using desired Bootstrap classes, such as `mb-3`, for formatting). This allows you to group form items, such as a `<label>` and its corresponding `<input>` control, together and provides responsive formatting.

b. Add the `form-label` class to each of the `<label>` elements, except for the checkbox label – use `form-check-label` for that.

c. Add the `form-control` class to each of the textual input controls. This class provides styles for general appearance, focus state, sizing, and more.

d. Wrap the checkbox and corresponding label elements in a with the `form-check` class, and use the `form-check-input` class for the checkbox.

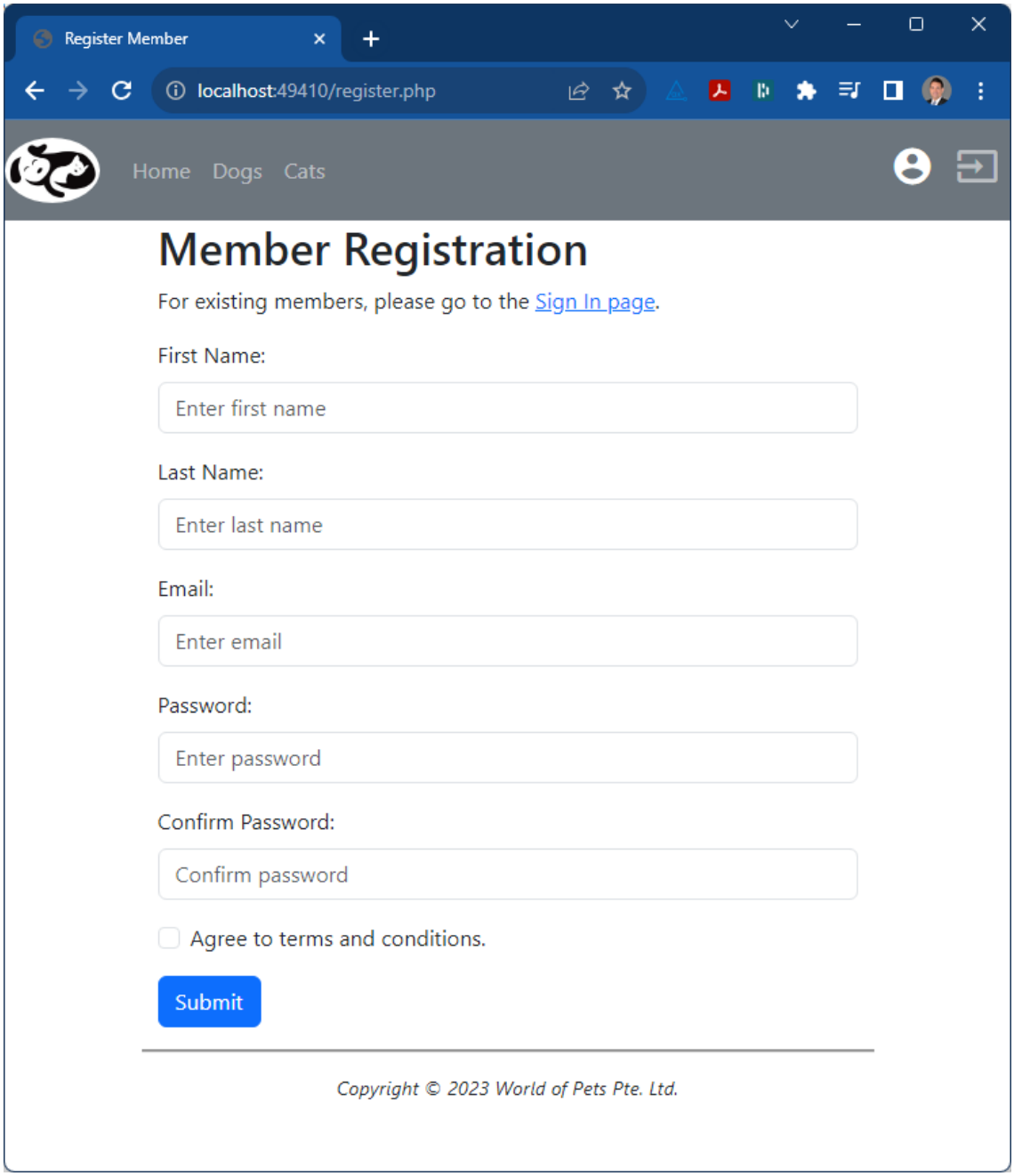
e. Add the `btn` and `btn-primary` classes to the submit `<button>` element.

f. For more details on formatting forms with Bootstrap, refer to: https://getbootstrap.com/docs/5.3/forms/overview/.

After implementing the above changes, your form code should be similar to this:

```
<form action="process_register.php" method="post">
<div class="mb-3">
    <label for="fname" class="form-label">First Name:</label>
    <input type="text" id="fname" name="fname" class="form-control"
        placeholder="Enter first name">
</div>
<div class="mb-3">
    <label for="lname" class="form-label">Last Name:</label>
    <input type="text" id="lname" name="lname" class="form-control"
        placeholder="Enter last name">
</div>
<div class="mb-3">
    <label for="email" class="form-label">Email:</label>
    <input type="email" id="email" name="email" class="form-control"
        placeholder="Enter email">
</div>
<div class="mb-3">
    <label for="pwd" class="form-label">Password:</label>
    <input type="password" id="pwd" name="pwd" class="form-control"
        placeholder="Enter password">
</div>
<div class="mb-3">
    <label for="pwd_confirm" class="form-label">Confirm Password:</label>
    <input type="password" id="pwd_confirm" name="pwd_confirm" class="form-control"
        placeholder="Confirm password">
</div>
<div class="mb-3 form-check">
    <input type="checkbox" name="agree" id="agree" class="form-check-input">
    <label class="form-check-label" for="agree">
        Agree to terms and conditions.
    </label>
</div>
<div class="mb-3">
    <button type="submit" class="btn btn-primary">Submit</button>
</div>
</form>
```

…and the registration page should now look and behave much better:



3.6 Lastly, now that we are adding additional pages to our website, you'll want to implement a method for highlighting the menu item corresponding to the page currently being viewed. For example, in the screenshot above, the icon for Member Registration is highlighted

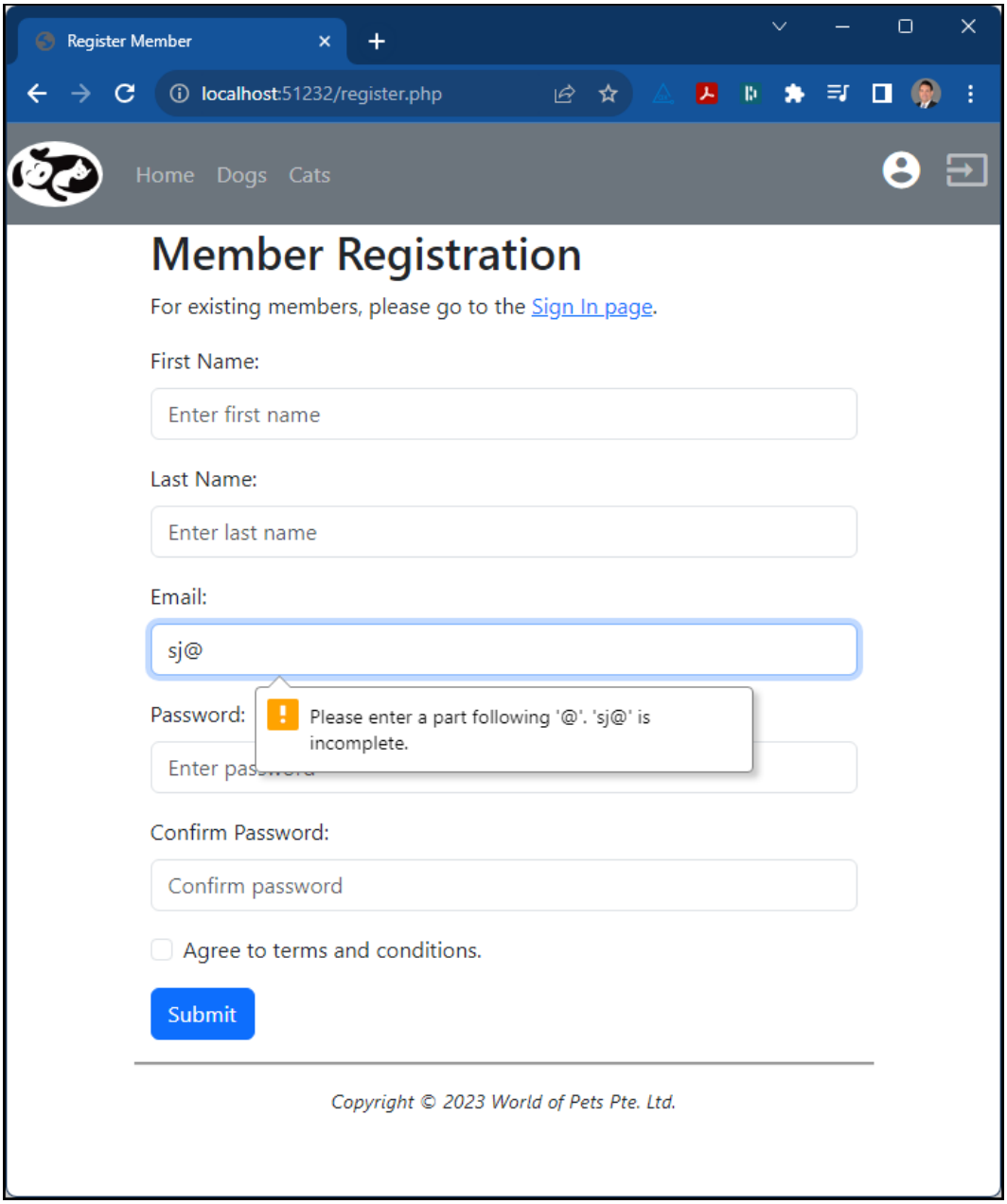
instead of the 'Home' menu item. There are many ways to do this using JavaScript and CSS (and jQuery) - below is one method:

```
/*
 * This function sets the currently selected menu item to the 'active' state.
 * It should be called whenever the page first loads.
 */
function activateMenu()
{
    const navLinks = document.querySelectorAll('nav a');
    navLinks.forEach(link =>
    {
        if (link.href === location.href)
        {
            link.classList.add('active');
        }
    })
}
```

4. EXERCISE 2: FORM VALIDATION

4.1 As it stands, there is no validation (other than the `type` attribute) for our form - users can submit empty or invalid input. Or even worse, a nefarious person could submit malicious code and hack our website! In this exercise, we'll use a combination of client-side and server-side form validation & sanitization to make our form more robust and secure.

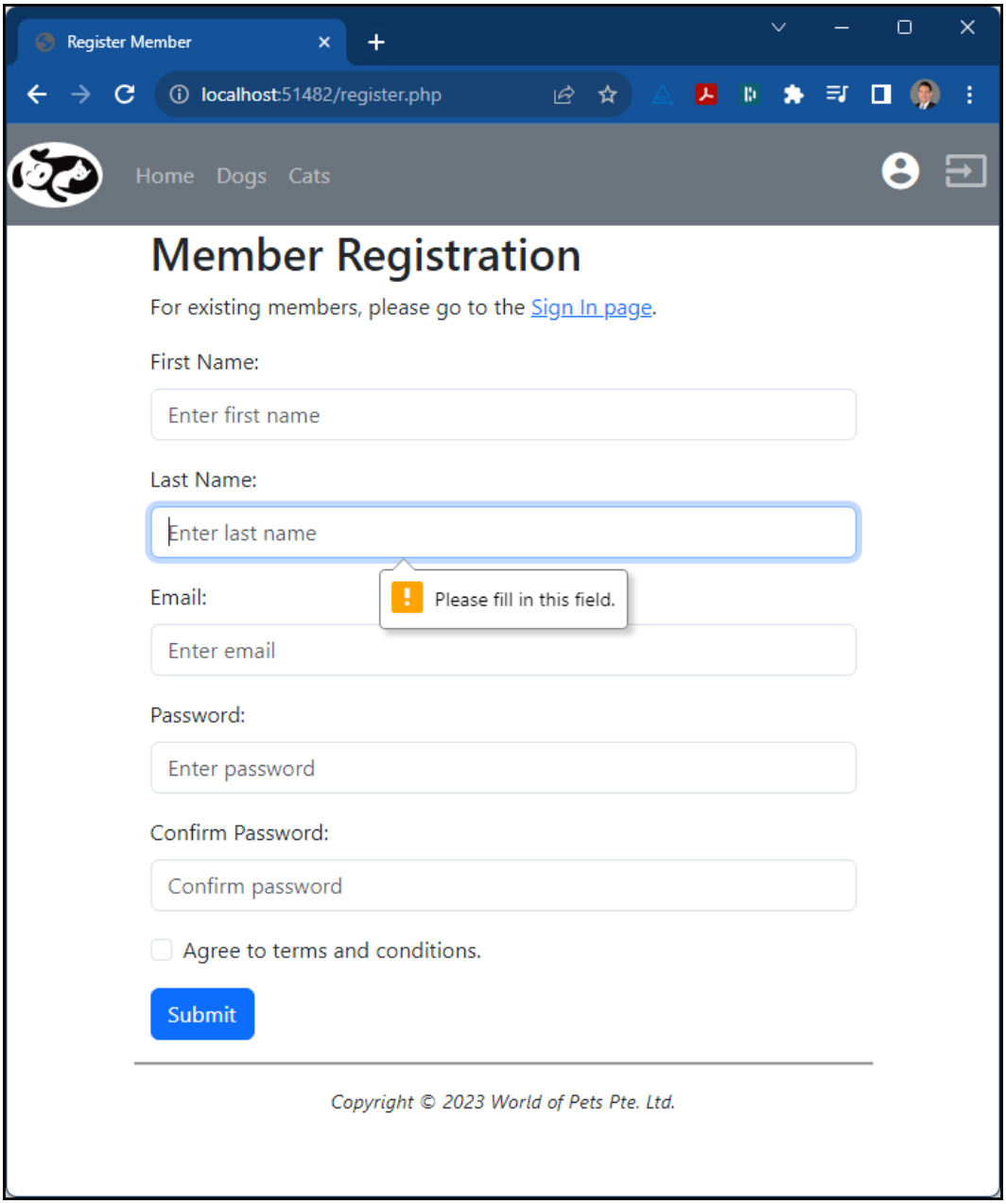
4.2 Let's start by adding some HTML5 attributes to help with validation. We've already used the `type` attribute, which indicates to the browser the type of input field. Not only does this attribute help make our form semantically correct and aid accessibility, it also can provide some validation depending on the type. For example, `type="email"` will tell the browser to do some basic checking to verify if the text is a properly formatted email address. And `type="password"` will instruct the browser to hide the characters being typed. Example:



4.3 Another attribute we can use is `required` - this specifies whether a form field needs to be filled in before the form can be submitted. In our case, we want to require all fields except for First Name, since it's possible for a person to have only one name (as is common in Indonesia). Insert the `required` attribute in all form elements except for the fname <input> and the <button>. Example:

```
<input required type="text" id="lname" name="lname" class="form-control"
placeholder="Enter last name">
```

Now if you try to submit the form without filling in the text boxes, you'll see an error message from the browser:



4.4 Another useful set of HTML5 attributes are `minlength` and `maxlength`. When applied to a textual <input> field, they restrict the number of characters allowed. This can help protect

against buffer overflow attacks. Let's set the maximum length of our name and email fields to 45 characters, e.g.:

```
<input required maxlength="45" type="text" id="lname" name="lname"
class="form-control" placeholder="Enter last name">
```

4.5 There are a few more attributes available to help with client-side form validation, but for our case, these should be sufficient. For more information, refer to: https://developer.mozilla.org/en-US/docs/Learn/Forms/Form_validation.

Note that you may also use JavaScript to perform more customized validation on the client if you have the need. Just be aware that JavaScript can be easily bypassed.

4.6 At this point, we have standard client-side form validation to ensure that the user does not leave any required fields blank, the email address is more or less formatted correctly, etc. The form is also responsive and provides immediate feedback to the user if they forgot to enter something.

However, this is only half the job - we must also perform *sanitization* in addition to validation, to protect against security breaches. Malicious users often employ an attack known as cross-site scripting (XSS) to inject client-side script into web pages viewed by others. SQL injection is another common exploit. Hijacking form submissions is a typical way to carry out such attacks, therefore we must **always think security** when processing PHP forms. In the remaining part of this exercise, we'll learn how to process forms securely on the server-side using PHP.

4.7 Create a new file called `process_register.php`. In this file, we'll write the PHP script that will process the registration form when it is submitted to the server. Here we can use the `$_POST[]` *superglobal* variable to retrieve the values submitted in the form.

Example: `$_POST["fname"]` would return the first name provided in the `<input name="fname">` field.

4.8 Consider the following PHP script which validates and sanitizes an email address submitted by the user via a form POST:

```php
<?php
$email = $errorMsg = "";
$success = true;

if (empty($_POST["email"]))
{
    $errorMsg .= "Email is required.<br>";
    $success = false;
}
else
{
    $email = sanitize_input($_POST["email"]);

    // Additional check to make sure e-mail address is well-formed.
    if (!filter_var($email, FILTER_VALIDATE_EMAIL))
    {
        $errorMsg .= "Invalid email format.";
        $success = false;
    }
}

if ($success)
{
    echo "<h4>Registration successful!</h4>";
    echo "<p>Email: " . $email;
}
else
{
    echo "<h4>The following input errors were detected:</h4>";
    echo "<p>" . $errorMsg . "</p>";
}

/*
 * Helper function that checks input for malicious or unwanted content.
 */
function sanitize_input($data)
{
    $data = trim($data);
    $data = stripslashes($data);
    $data = htmlspecialchars($data);
    return $data;
}
?>
```

This “bare bones” example only illustrates validation of the ‘email’ form field, but you should be able to extend this pattern to validate the remaining input fields. Some interesting things to take note of:

a. The function `sanitize_input()` is used to filter any malicious or unwanted characters from the form input. As a rule, **NEVER accept user input "as-is"** – you should always employ a filtering method to make sure your website is not victimized by hackers. However, take note of the special instructions for handling passwords as mentioned in the next section.
b. The `filter_var()` PHP function is used to further qualify the email address to make sure it matches standard formatting. `filter_input()` is another useful PHP function that you can look into as well.

4.9 Using the above example as a guide, fill in the remaining code to validate all of the form data on the server using PHP. You can temporarily disable your client-side form validation in order to verify that the server-side validation is working. At a minimum, you should:

a. Make sure required fields such as last name, email and password, have been provided.

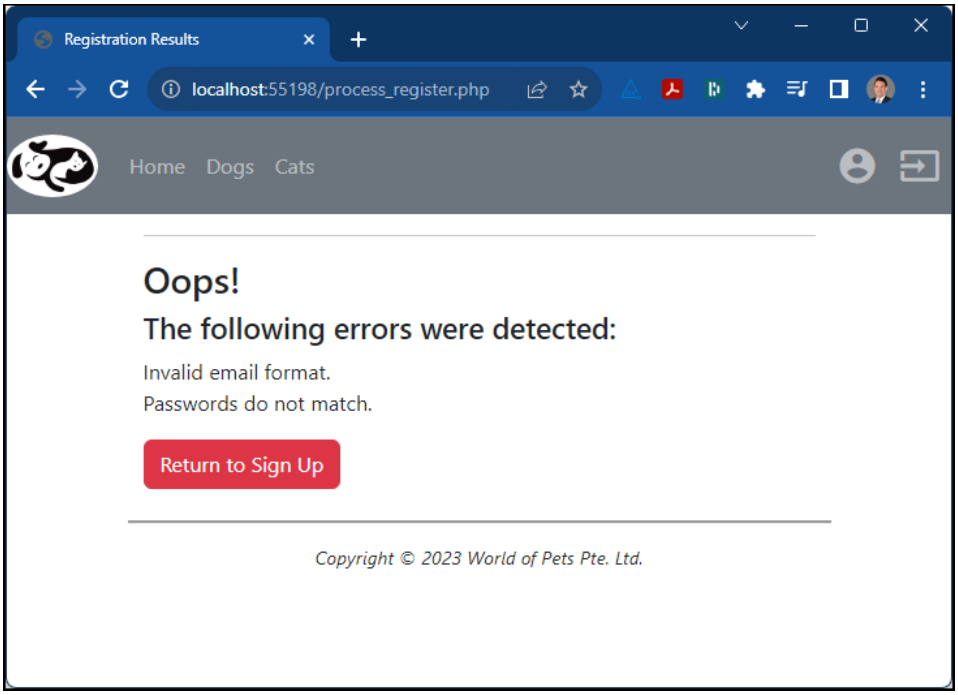
b. Verify that the email address is properly formatted and doesn't contain invalid characters.

c. Make sure the password matches the confirmation password.

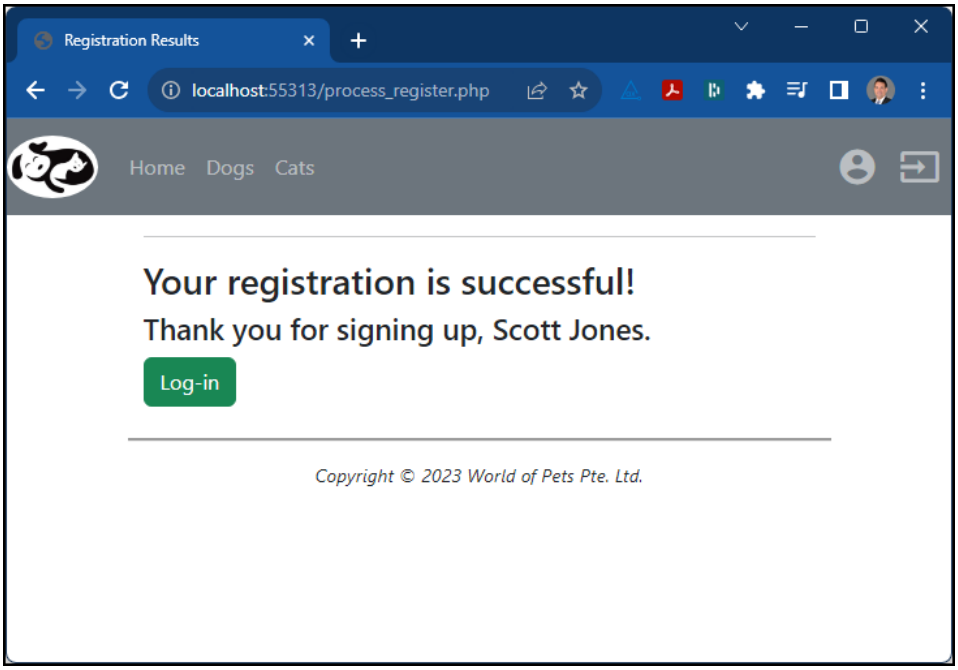
d. All input *with the exception of the password* should be sanitized/filtered to protect against malicious attacks. For the password, we generally do not sanitize as it could strip out characters that are meant to be part of the password. Instead, you should hash the password using the PHP function `password_hash()` and be sure to NEVER output the plaintext password to the web page.

4.10 Lastly, you can add logic, styling, and informative wording to the `process_register.php` page so it is more presentable and informative to the user.

Here's an example with error messages:



And here's an example of successful form submission:



4.11 In a future Lab, we'll close the loop by implementing database functionality for saving the user registration data, and add the log-in page.

Discussion Questions:

a. Why do we do validation on *both* the client and the server sides? What are the plusses and minuses of each?

b. Some web developers will insist that sanitization should only be performed on output, never on input, so as to preserve the user's original input as submitted. What are your thoughts on this?

c. There are two ways that form data can be submitted to the server: **POST** and **GET**. What are the differences, advantages and disadvantages of these two methods? Why did we choose the POST method for our form? **Tip**: go to https://www.w3schools.com/html/html_forms.asp and read the section titled "The Method Attribute".

5. SUBMISSION OF LAB ASSIGNMENT

5.1 In order to receive credit for this Lab assignment, you must submit your completed work to xSiTe LMS before the end of the Lab session. To submit your work:

a. Save all files and close VS Code.

b. In File Explorer, navigate to the location where you saved your project and right-click on the folder name, then select 'Send to -> Compressed (zipped) folder' and ZIP up your entire project. **Note: only .zip format is acceptable, do not use .rar, .7z, or any other format.**

c. In the INF1005 module on xSiTe, go to **Assessments->DropBox** and locate the Dropbox folder corresponding to this Lab. Click the link to open the Dropbox then hit the **Add a File** button to submit your .zip file. You may also add comments if desired. Be sure to hit **Submit** to complete your submission.

d. Remember to save a copy of your work as we will be building upon this website in subsequent Lab assignments.

6. ADDITIONAL PRACTICE

6.1 Once you've completed this Lab assignment, you are encouraged to try out the following online tutorials:

a. HTML Forms: https://www.w3schools.com/html/html_forms.asp

b. Client-side Form Validation: https://developer.mozilla.org/en-US/docs/Learn/Forms/Form_validation

c. Bootstrap Form Validation: https://getbootstrap.com/docs/5.3/forms/validation/

d. PHP Form Handling: https://www.w3schools.com/php/php_forms.asp

e. PHP Form Validation: https://www.w3schools.com/php/php_form_validation.asp

f. PHP Filters: https://www.w3schools.com/php/php_filter.asp